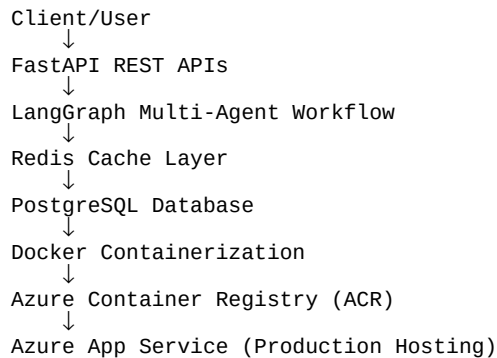


Telecom AI Incident Agent – Enterprise Revision Handbook

Enterprise Architecture Overview



Version 1 – FastAPI + LangGraph Setup

Goal:
Build a telecom incident analysis backend capable of receiving telecom network alerts and generating AI-

Tools Used:

- Python
- FastAPI
- Uvicorn
- LangGraph
- Pydantic

Why We Used FastAPI:

FastAPI provides high-performance asynchronous APIs and automatic Swagger documentation.

Important Code:

```
@app.get("/health")
def health():
    return {"status": "healthy"}
```

Why This Code Was Used:

This endpoint verifies whether the application is running correctly.

Command Used:

```
uvicorn app.main:app --reload
```

Why We Used Uvicorn:

Uvicorn is an ASGI server optimized for FastAPI applications.

Version 2 – PostgreSQL Integration

Goal:

Persist telecom incidents inside a relational database.

Why PostgreSQL:

PostgreSQL is enterprise-grade, scalable, and works well with SQLAlchemy.

Important Code:

```
engine = create_engine(DATABASE_URL)
```

Why This Was Used:

This creates the SQLAlchemy database engine responsible for database communication.

Problem Solved:

Before database integration, all incident data disappeared after API execution.
After PostgreSQL integration, incident history became persistent.

Commands:

```
docker run --name postgres -p 5432:5432 postgres
```

Real Learning:

Applications require persistent storage in production systems.

Version 3 – Redis Caching

Goal:

Improve API performance by caching repeated telecom alerts.

Why Redis:

Redis is an in-memory datastore optimized for high-speed caching.

Important Code:

```
redis_client.set(cache_key, json.dumps(result))
```

Why This Was Used:

Repeated telecom alerts do not need recalculation every time.

Real-World Benefit:

Reduces response time and improves scalability.

Enterprise Understanding:

Caching reduces computational load and improves system throughput.

Version 4 – Multi-Agent AI Workflow

Goal:

Create enterprise-style telecom AI workflows.

Agents Created:

- SeverityAnalysisAgent
- RootCauseAnalysisAgent
- RecommendationAgent
- EscalationDecisionAgent
- LLMSummaryAgent

Why LangGraph:

LangGraph helps orchestrate multiple AI workflow steps sequentially.

Business Logic:

Each AI agent performs a specialized telecom operational responsibility.

Example:

SeverityAnalysisAgent determines whether the issue is low, medium, or critical.

Version 5 – Dockerization

Goal:

Containerize the application for consistent deployment.

Why Docker:

Docker packages application code, dependencies, runtime, and configurations together.

Important Files:

- Dockerfile
- docker-compose.yml

Important Code:
FROM python:3.11

Why This Was Used:
Provides standardized Python runtime environment.

docker-compose.yml Purpose:
Runs multiple services:
- FastAPI
- Redis
- PostgreSQL

Real Learning:
Containers solve "works on my machine" issues.

Version 6 – Health & Readiness APIs

Goal:
Implement production-grade monitoring APIs.

Endpoints:
/health
/ready

Why These APIs Matter:
Cloud platforms use health checks to determine application availability.

Example:
GET /health

Response:
{
 "status": "healthy"
}

Enterprise Importance:
Health monitoring is mandatory in production cloud systems.

Version 7 – Azure Cloud Deployment

Goal:
Deploy the telecom AI platform into Microsoft Azure cloud.

Azure Services Used:
- Azure CLI
- Azure Resource Group
- Azure Container Registry
- Azure App Service
- Azure Web App

Step-by-Step Cloud Deployment:

1. Azure Login
az login

Purpose:
Authenticate local machine with Azure cloud.

2. Create Resource Group
az group create --name telecom-ai-rg --location eastus2

Purpose:
Resource Group acts as project container inside Azure.

3. Create Azure Container Registry
az acr create --resource-group telecom-ai-rg --name telecomaiacr4383 --sku Basic --location eastus2
Purpose:

Store Docker images inside Azure cloud.

4. Push Docker Image

```
docker tag  
docker push
```

Purpose:

Upload local Docker image into Azure cloud registry.

5. Create Azure App Service Plan

```
az appservice plan create
```

Purpose:

Provision compute resources for hosting containerized application.

6. Create Azure Web App

```
az webapp create
```

Purpose:

Create public cloud-hosted application endpoint.

Major Azure Troubleshooting:

Problem:

Azure Container Apps failed due to subscription region restrictions.

Solution:

Changed regions and registered Microsoft.ContainerRegistry provider.

Problem:

Docker container crashed in Azure.

Root Cause:

host.docker.internal works locally but not in Azure cloud.

Solution:

Made database startup optional using try-except.

Why Azure Deployment Matters:

Localhost runs only on personal laptop.

Azure deployment runs application on Microsoft cloud servers accessible publicly.

Production URL:

<https://telecom-ai-agent-4383.azurewebsites.net/docs>

Localhost vs Azure Cloud

Localhost:

- Runs only on personal machine
- Stops when laptop shuts down
- Not publicly accessible

Azure Cloud:

- Runs inside Microsoft data centers
- Publicly accessible
- Scalable and production-ready
- Enterprise deployment infrastructure

Biggest Learning:

Cloud deployment converts local code into real production software.

Future Scope

Future Enhancements:

- JWT Authentication
- Kafka Streaming
- Azure Redis Cache

- Azure PostgreSQL
- Kubernetes (AKS)
- Monitoring Dashboards
- Prometheus & Grafana
- CI/CD GitHub Actions
- Real LLM APIs